

Examen Parcial Plataforma de Créditos

Stack: ASP.NET Core MVC (.NET 10) + Identity, EF Core, Razor Views, Session/Cache con Redis

Infra: Render.com (Web Service) + Redis gestionado (<https://app.redislabs.com/>)

Control de versiones: GitHub con ramas por pregunta y PRs hacia main

Tiempo de la versión original: 2 horas

Extensión práctica: WebSocket con ASP.NET PieSocket y Cloud MQ con RabbitMQ gestionado en CloudAMQP.

Tiempo sugerido para la versión ampliada: 3 horas y 30 minutos, con cuentas y conexiones preparadas previamente.

WebSocket: <https://piehost.com/>

Queue: <https://customer.cloudamqp.com/>

Memoria Temporal: <https://cloud.redis.io/#/>

Search Engine: <https://dashboard.algolia.com/>

Infra: <https://render.com/>

Contexto

Una entidad financiera necesita una plataforma web interna para gestionar solicitudes de crédito de sus clientes. El sistema debe permitir que los usuarios autenticados registren solicitudes de crédito, las cuales serán evaluadas por analistas de riesgo bajo reglas básicas de negocio.

El sistema debe garantizar que no se exceda la capacidad de pago del cliente y que solo exista una solicitud activa por cliente, mientras los analistas pueden aprobar o rechazar dichas solicitudes.

El objetivo de la evaluación es medir la capacidad de:

- Trabajar con MVC e Identity.
 - Modelar datos con restricciones en EF Core/Sqlite.
 - Usar sesiones y Redis para mejorar experiencia y rendimiento.
 - Implementar validaciones de negocio en servidor.
 - Versionar con Git (ramas, merges, PRs).
 - Desplegar en Render.com con configuración de variables de entorno.
 - Notificar cambios de estado en tiempo real mediante WebSocket.
 - Publicar y consumir mensajes con una cola en la nube.
-

Reglas generales

- Cada pregunta se resuelve en una rama distinta.
- No está permitido trabajar directamente en main.
- Cada rama debe cerrarse con un PR hacia main.
- README obligatorio con pasos locales, migraciones, variables de entorno y URL de Render.
- Tiempo de la versión original: 2 horas; ampliación sugerida: 90 minutos.
- Las preguntas 6 y 7 deben tener ramas y PRs separados. Crear cada rama desde main actualizado, después de integrar sus dependencias.
- Tiempo adicional sugerido: 45 minutos por cada nueva pregunta.

- Preparar antes del examen las cuentas de Redis, Render y CloudAMQP; no subir credenciales al repositorio.
-

Pregunta 1 — Bootstrap + Modelo de datos

Rama: feature/bootstrap-dominio

Requerido

Crear proyecto base con dotnet new mvc --auth Individual.

Configurar EF Core con SqlLite.

Modelos:

Cliente: Id, Usuarioid, IngresosMensuales, Activo.

SolicitudCredito: Id, Clienteld, MontoSolicitado, FechaSolicitud, Estado {Pendiente, Aprobado, Rechazado}, MotivoRechazo.

Restricciones:

IngresosMensuales > 0.

MontoSolicitado > 0.

- Un cliente solo puede tener una solicitud en estado Pendiente.
 - No se puede aprobar una solicitud si el MontoSolicitado es mayor a 5 veces los IngresosMensuales.
 - Configuración inicial con al menos 2 clientes, 2 solicitudes (una pendiente y una aprobada) y un usuario con rol Analista.
-

Pregunta 2 — Catálogo de solicitudes y filtros

Rama: feature/catalogo-solicitudes

Requerido

Vista “Mis solicitudes” con listado de solicitudes del usuario autenticado.

Filtros: Estado, rango de MontoSolicitado, rango de fechas.

Vista detalle con información completa de la solicitud y estado.

Validaciones server-side:

No aceptar montos negativos.

No aceptar rangos de fechas inválidos (fecha inicio mayor a fecha fin).

Pregunta 3 — Registro y validaciones de solicitud

Rama: feature/solicitudes

Requerido

Formulario de registro de solicitud: al enviar se crea una SolicitudCredito en estado Pendiente.

Validaciones server-side:

Usuario debe estar autenticado.

Cliente debe estar activo.

No permitir más de una solicitud Pendiente por cliente.

El monto solicitado no puede superar 10 veces los ingresos mensuales.

Feedback claro en la misma vista (éxito/error).

Pregunta 4 — Sesiones y Redis

Rama: feature/sesion-redis

Requerido

Sesión (Redis-backed):

Guardar la última solicitud visitada para mostrar en el layout un enlace “Ver última solicitud {Monto}”.

Cache (Redis):

Cachear por 60s el listado de solicitudes del usuario autenticado.

Invalidar cuando:

Se registre una nueva solicitud.

Se actualice el estado de una solicitud.

Pregunta 5 — Panel de Analista (rol)

Rama: feature/panel-analista

Requerido

Crear rol Analista.

Panel /Analista con [Authorize(Roles="Analista")]:

Lista de solicitudes en estado Pendiente.

Opción de Aprobar solicitud.

Opción de Rechazar solicitud (requiere MotivoRechazo).

Validaciones:

No aprobar si el monto excede 5 veces los ingresos.

No procesar solicitudes ya aprobadas o rechazadas.

Motivo obligatorio en rechazo.

Acceso denegado a usuarios sin rol.

Pregunta 6 — Notificaciones con WebSocket

Rama: feature/websocket-notificaciones

Objetivo: el cliente debe conocer el resultado de la evaluación sin actualizar manualmente la página.

Requerido

- Implementar un Hub de ASP.NET con PiSocket en /hubs/solicitudes, protegido con autenticación de Identity. Usar WebSocket como transporte y verificarlo en las herramientas del navegador.
- Conectar las vistas “Mis solicitudes” y detalle. Cuando un analista apruebe o rechace, enviar el evento SolicitudEstadoActualizado con SolicitudId, Estado y MotivoRechazo, únicamente al usuario propietario.
- Guardar primero el estado en la base de datos e invalidar su caché Redis; después emitir el evento. Actualizar el estado mostrado y presentar un aviso sin recargar la página.
- Obtener la identidad del usuario desde el servidor; no aceptar un UsuarioId enviado por el navegador para elegir al destinatario. Mantener las validaciones y autorización del panel Analista.

- Mostrar el estado de conexión e implementar reconexión. Al reconectar, consultar el estado vigente desde el servidor para recuperar cambios ocurridos durante la desconexión.

Prueba y evidencias

Abrir sesiones independientes de un cliente y un analista. Aprobar o rechazar una solicitud y comprobar el cambio inmediato en la sesión del cliente. Con un segundo cliente, comprobar que no recibe el evento. Adjuntar captura de la conexión WebSocket y del resultado; una conexión anónima al Hub debe rechazarse.

Pregunta 7 — Mensajería asíncrona con Cloud MQ

Rama: feature/cloudmq-notificaciones

Objetivo: practicar productor, cola y consumidor generando una notificación de recepción de solicitud. Para este examen, Cloud MQ se implementa con RabbitMQ gestionado en CloudAMQP.

Requerido

- Declarar la cola durable solicitudes.notificaciones y conectarse por AMQPS desde el servidor. Configurar credenciales mediante variables de entorno.
- Después de guardar una nueva solicitud Pendiente, publicar un mensaje JSON persistente de tipo SolicitudRegistrada con MessageId (UUID), SolicitudId, UsuarioId y FechaEventoUtc. No publicar si falla la validación o la persistencia de la solicitud.
- Implementar un consumidor con BackgroundService y RabbitMQ.Client. Al recibir el mensaje, guardar en SQLite una Notificacion: Id, MessageId, SolicitudId, UsuarioId, Texto y FechaProcesamientoUtc. Texto sugerido: "Recibimos tu solicitud de crédito y está pendiente de evaluación".
- Mostrar las notificaciones en una vista "Mis notificaciones" filtrada por el usuario autenticado. No se requiere enviar correos ni SMS reales. El consumidor no aprueba ni rechaza créditos.
- Confirmar el mensaje con ACK manual solamente después de guardar la notificación. Establecer unicidad de MessageId para que una redelivery no genere duplicados; si ya fue procesado, confirmar sin insertar nuevamente.
- Usar confirmación del publicador para verificar aceptación por el broker. Ante una falla de publicación, conservar la solicitud, registrar el error y advertir que la notificación no pudo encolarse. Documentar el procedimiento de reenvío con el mismo MessageId; no es obligatorio implementar un patrón outbox.
- Si falla el procesamiento, registrar el error y no confirmar como exitoso. Evitar reintentos infinitos: documentar el reenvío manual para esta práctica. Un mensaje inválido debe rechazarse sin reencolar y dejar evidencia en logs.

Prueba y evidencias

Desactivar el consumidor con RabbitMq__ConsumerEnabled=false, registrar una solicitud y mostrar el mensaje pendiente en CloudAMQP. Reactivarlo y verificar que la cola se vacía y aparece una sola notificación. Reenviar el mismo MessageId y comprobar que no se duplica. Adjuntar capturas y pasos reproducibles al README.

Relación entre las prácticas

WebSocket comunica el resultado de la evaluación al navegador conectado. Cloud MQ desacopla el registro de la solicitud del procesamiento de su notificación de recepción. Cada práctica debe poder demostrarse por separado y tener su propio PR hacia main.

Pregunta 8 — Despliegue en Render

Rama: deploy/render

Requerido

Desplegar en Render como Web Service.

Variables mínimas:

ASPNETCORE_ENVIRONMENT=Production

ASPNETCORE_URLS=http://0.0.0.0:\${PORT}

ConnectionStrings__DefaultConnection

Redis__ConnectionString

RabbitMq__ConnectionString (URI amqps del proveedor)

RabbitMq__QueueName=solicitudes.notificaciones

RabbitMq__ConsumerEnabled=true

Verificar online: login, registro de solicitud, validaciones, panel analista y uso de cache/sesión.

Verificar también WebSocket seguro (wss), publicación y consumo en Cloud MQ. Para esta práctica, ejecutar una sola instancia del Web Service y el consumidor como BackgroundService dentro de ella. Documentar cómo se conserva SQLite entre despliegues y reinicios. Configurar el comando de inicio para expandir PORT; no asumir que \${PORT} se expande dentro de una variable de entorno.

Criterios de corrección

Uso correcto de ramas, merges y PRs en GitHub.

Validaciones implementadas en servidor (no solo cliente).

Sesiones y cache Redis funcionando en Render.

Despliegue accesible con README bien documentado.

Notificación WebSocket dirigida al propietario correcto, sin recargar la página.

Publicación real en Cloud MQ, consumo con confirmación manual y control de duplicados.

Evidencias reproducibles de ambas prácticas y PR independiente para cada una.

Referencias técnicas

CloudAMQP: <https://www.cloudamqp.com/>

Confirmaciones de RabbitMQ: <https://www.rabbitmq.com/docs/confirmations>